



## Lab Activity



Week 12: Introduction to Functions - Building Reusable Code



Duration: 60 minutes — Work in pairs!

Name: \_\_\_\_\_

Partner: \_\_\_\_\_

### Lab Goals

By the end of this lab, you will be able to:

- Define functions using `def` with proper syntax
- Pass information to functions using parameters
- Return values from functions and use them in your program
- Understand local vs global scope using memory diagrams
- Debug functions by tracing execution flow

### Getting Started

- Extract the given [week12\\_lab.zip](#) file to your desktop
- **KEY CONCEPT:** Functions are reusable blocks of code!
- Remember: `def function_name(parameters):` with a colon and indentation

## 1 Exercise 1: Your First Functions (8 minutes)

Learn to define and call simple functions!

Open [week12\\_ex1.py](#) and complete:

```
1  # Define a simple greeting function
2  def greet():
3      print("Assalam-o-Alaikum!")
4      print("Welcome to Python functions!")
5
6  # Call the function - notice the parentheses!
7  greet()
8
9  # Define a function that prints a separator line
10 def print_line():
11     print("=" * 40)
12
13 # Use it to organize output
14 print_line()
15 print("This is a section")
16 print_line()
17
18 # Practice: Create a function called show_menu
19 def _____(): # Fill in function name
```

```

20     print("\n--- Calculator Menu ---")
21     print("1. Add")
22     print("2. Subtract")
23     print("3. Multiply")
24     print("4. Quit")
25     print("-" * 23)
26
27     # Call your function
28     -----() # Fill in function call
29
30     # What happens if you forget parentheses?
31     print("\nWithout parentheses:")
32     print(greet) # This prints the function object, not calling it!

```

 **Tip:** Always use parentheses () when calling a function, even with no parameters!

 **Checkpoint:** Can you define and call functions? Show teacher!

## 2 Exercise 2: Functions with Parameters (10 minutes)

Make functions flexible by passing information to them!

Open [week12\\_ex2.py](#) and complete:

```

1  # Function with one parameter
2  def greet_person(name):
3      print(f"Assalam-o-Alaikum, {name}!")
4
5  # Call with different arguments
6  greet_person("Ahmed")
7  greet_person("Fatima")
8  greet_person("Ali")
9
10 # Function with multiple parameters
11 def introduce(name, age, city):
12     print(f"{name} is {age} years old and lives in {city}")
13
14 introduce("Sara", 13, "Karachi")
15 introduce("Omar", 14, "Lahore")
16
17 # Practice: Create a function to display student info
18 def show_student(name, grade, subject):
19     print(f"\n--- Student Card ---")
20     print(f"Name: {_____}") # Fill in parameter
21     print(f"Grade: {_____}") # Fill in parameter
22     print(f"Subject: {_____}") # Fill in parameter
23     print("-" * 20)
24
25 # Call your function
26 show_student("Aisha", "7th", "Computer Science")
27 show_student(_____, _____, _____) # Add your own info!
28
29 # Order matters! What's wrong here?
30 def calculate_rectangle(length, width):
31     area = length * width
32     perimeter = 2 * (length + width)

```

```

33     print(f"Length: {length}, Width: {width}")
34     print(f"Area: {area}, Perimeter: {perimeter}")
35
36     # These give different results!
37     print("\nFirst call:")
38     calculate_rectangle(10, 5)
39     print("\nSecond call:")
40     calculate_rectangle(5, 10)

```

 **Pair Programming:** Switch who types every 5 minutes!

 **Checkpoint:** Can you pass parameters to functions? Excellent!

### 3 Exercise 3: Return Values (10 minutes)

Learn the difference between printing and returning!

Open [week12\\_ex3.py](#) and complete:

```

1  # Function that PRINTS (shows on screen but returns None)
2  def add_print(a, b):
3      print(a + b)
4
5  # Function that RETURNS (sends value back)
6  def add_return(a, b):
7      return a + b
8
9  # See the difference!
10 print("=== PRINT vs RETURN ===")
11 result1 = add_print(3, 4)    # Prints 7
12 result2 = add_return(3, 4)   # Returns 7
13
14 print(f"\nresult1 = {result1}")    # None!
15 print(f"result2 = {result2}")      # 7!
16
17 # Why does this matter?
18 print("\n=== Using returned values ===")
19
20 # This WORKS - we can use the returned value
21 total = add_return(10, 20) + add_return(5, 5)
22 print(f"Total: {total}")
23
24 # This DOESN'T WORK as expected
25 # total = add_print(10, 20) + add_print(5, 5) # Error!
26
27 # Practice: Convert this print function to a return function
28 def calculate_average_print(num1, num2, num3):
29     avg = (num1 + num2 + num3) / 3
30     print(avg)    # This prints but returns None
31
32 def calculate_average_return(num1, num2, num3):
33     avg = (num1 + num2 + num3) / 3
34     ----- avg    # Fill in: return
35
36 # Test your function
37 quiz = 85

```

```

38 homework = 90
39 exam = 78
40
41 average = calculate_average_return(quiz, homework, exam)
42 print(f"\nYour average: {average:.1f}")
43
44 # Now we can use it in calculations!
45 if average >= 80:
46     print("Great job! You got an A!")
47 elif average >= 70:
48     print("Good work! You got a B!")
49 else:
50     print("Keep studying!")
51
52 # Functions can return early
53 def check_pass(score):
54     if score >= 50:
55         return "PASS"
56     return "FAIL" # Only runs if score < 50
57
58 print(f"\nScore 75: {check_pass(75)}")
59 print(f"Score 45: {check_pass(45)}")

```

 **Tip:** Use `return` when you need the value later. Use `print` when you just want to display something.

 **Checkpoint:** Do you understand `return` vs `print`? This is crucial!

## 4 Exercise 4: Function Scope - Memory Diagrams (12 minutes)

Understand where variables live and why it matters!

Open [week12\\_ex4.py](#) and complete:

```

1 # Global variable
2 score = 100
3
4 def update_score():
5     # This creates a LOCAL variable named score
6     score = 50 # Different from global score!
7     print(f"Inside function: score = {score}")
8
9 print(f"Before function call: score = {score}")
10 update_score()
11 print(f"After function call: score = {score}")
12
13 # The global score is unchanged!

```

 **Memory Diagram:** Draw the memory diagram for the code above:

**Global Frame**

score = \_\_\_\_\_

**update\_score() Frame**

score = \_\_\_\_\_

(Local frame disappears after function returns)

Continue in [week12\\_ex4.py](#):

```

1  # The RIGHT way: Use parameters and return values!
2  print("\n=== CORRECT APPROACH ===")
3
4  player_score = 100
5
6  def add_bonus(current_score, bonus_points):
7      # Parameters are LOCAL to the function
8      new_score = current_score + bonus_points
9      return new_score # Send the result back
10
11 # Update the global variable with the returned value
12 player_score = add_bonus(player_score, 25)
13 print(f"After bonus: {player_score}")
14
15 player_score = add_bonus(player_score, 10)
16 print(f"After another bonus: {player_score}")
17
18 # Practice: Fix this broken function
19 def broken_double():
20     number = number * 2 # ERROR: number doesn't exist locally!
21     return number
22
23 # Fixed version:
24 def fixed_double(number): # Take number as parameter
25     result = number * 2
26     ----- result # Fill in: return
27
28 x = 5
29 x = fixed_double(x)
30 print(f"\nDoubled: {x}") # Should be 10
    
```

 **Pair Programming:** Switch who types every 5 minutes!

 **Checkpoint:** Can you draw memory diagrams for scope? Show teacher!

## 5 Exercise 5: Building a Function Library (10 minutes)

Create reusable functions for common tasks!

Open [week12\\_ex5.py](#) and complete:

```

1  # === MATH FUNCTIONS ===
2
3  def square(n):
4      """Returns the square of n"""
    
```

```

5     return n * n
6
7 def cube(n):
8     """Returns the cube of n"""
9     return n * n * n
10
11 def is_even(n):
12     """Returns True if n is even, False otherwise"""
13     return n % 2 == 0 # Fill in: 0
14
15 def is_positive(n):
16     """Returns True if n is positive"""
17     return n > 0 # Fill in: >
18
19 # Test the math functions
20 print("== Math Functions ==")
21 print(f"square(5) = {square(5)}")
22 print(f"cube(3) = {cube(3)}")
23 print(f"is_even(4) = {is_even(4)}")
24 print(f"is_even(7) = {is_even(7)}")
25
26 # == STRING FUNCTIONS ==
27
28 def shout(text):
29     """Returns text in uppercase with exclamation"""
30     return text.upper() + "!"
31
32 def whisper(text):
33     """Returns text in lowercase with ellipsis"""
34     return text.lower() + "..."
35
36 def repeat_text(text, times):
37     """Returns text repeated 'times' times"""
38     return text * times # Fill in: times
39
40 # Test string functions
41 print("\n== String Functions ==")
42 print(shout("hello"))
43 print(whisper("BE QUIET"))
44 print(repeat_text("Python! ", 3))
45
46 # == GRADE FUNCTIONS ==
47
48 def calculate_percentage(obtained, total):
49     """Calculate percentage score"""
50     return (obtained / total) * 100
51
52 def get_grade(percentage):
53     """Convert percentage to letter grade"""
54     if percentage >= 90:
55         return "A+"
56     elif percentage >= 80:
57         return "A"
58     elif percentage >= 70:
59         return "B"
60     elif percentage >= 60:
61         return "C"
62     elif percentage >= 50:
63         return "D"
64     else:
65         return "F"

```

```

66
67 # Practice: Create a function that uses both!
68 def show_result(name, obtained, total):
69     """Display student's result with grade"""
70     pct = calculate_percentage(____, ____ ) # Fill in parameters
71     grade = get_grade(____) # Fill in parameter
72     print(f"{name}: {obtained}/{total} = {pct:.1f}% (Grade: {grade})")
73
74 # Test your function
75 show_result("Ahmed", 85, 100)
76 show_result("Sara", 72, 100)
77 show_result("Ali", 45, 100)

```

✔ **Checkpoint:** Function library complete? You're becoming a real programmer!

## 6 Exercise 6: Debug Detective (10 minutes)

Find and fix the errors in these functions!

Open [week12\\_ex6.py](#) and complete:

```

1 # Bug 1: Missing colon
2 def say_hello() # What's missing?
3     print("Hello!")
4
5 # Fix:
6 def say_hello():
7     print("Hello!")
8
9 # Bug 2: Wrong indentation
10 def calculate_sum(a, b):
11     result = a + b # What's wrong here?
12     return result
13
14 # Fix:
15 def calculate_sum(a, b):
16     result = a + b
17     ----- result # Fill in
18
19 # Bug 3: Forgetting to call the function
20 def show_greeting():
21     print("Welcome!")
22
23 show_greeting # This doesn't call it!
24
25 # Fix:
26 show_greeting() # Add -----
27
28 # Bug 4: Using print instead of return
29 def multiply(x, y):
30     print(x * y)
31
32 answer = multiply(5, 3)
33 total = answer + 10 # This will ERROR because answer is None!
34
35 # Fix:

```

```

36 def multiply_fixed(x, y):
37     ----- x * y # Fill in: return
38
39 answer = multiply_fixed(5, 3)
40 total = answer + 10
41 print(f"Total: {total}")
42
43 # Bug 5: Parameter order confusion
44 def subtract(a, b):
45     return a - b
46
47 # This should give 10 - 3 = 7, but...
48 result = subtract(3, 10) # Gives -7!
49 print(f"3 - 10 = {result}")
50
51 # Fix: Call with correct order
52 result = subtract(____, ____ ) # Fill in: 10, 3
53 print(f"10 - 3 = {result}")
54
55 # Bug 6: Scope error
56 total = 0
57
58 def add_to_total(amount):
59     total = total + amount # ERROR: can't access global 'total'
60     return total
61
62 # Fix using parameter:
63 def add_to_total_fixed(current_total, amount):
64     new_total = current_total + amount
65     return new_total
66
67 total = 0
68 total = add_to_total_fixed(total, 50)
69 total = add_to_total_fixed(total, 30)
70 print(f"Final total: {total}") # Should be 80

```

 **Tip:** When debugging functions, check: (1) colon after def, (2) indentation, (3) parentheses in call, (4) return vs print, (5) parameter order!

 **Checkpoint:** All bugs found and fixed? Excellent debugging skills!

## 7 Lab Summary

 **Checkpoint:** Final checkpoint - make sure you have:

- ☐ Defined functions with proper syntax (def, colon, indent)
- ☐ Called functions with parentheses
- ☐ Passed parameters to make functions flexible
- ☐ Used return to send values back from functions
- ☐ Understood the difference between print and return



- ☐ Drew memory diagrams showing local vs global scope
- ☐ Built a library of reusable functions
- ☐ Debugged common function errors

### Reflection (2 minutes)

Rate your understanding of today's concepts:

| Concept                  | 😞 Need Help           | 😬 Getting It          | 😄 Got It!             |
|--------------------------|-----------------------|-----------------------|-----------------------|
| Defining functions       | <input type="radio"/> | <input type="radio"/> | <input type="radio"/> |
| Parameters and arguments | <input type="radio"/> | <input type="radio"/> | <input type="radio"/> |
| Return vs print          | <input type="radio"/> | <input type="radio"/> | <input type="radio"/> |
| Local vs global scope    | <input type="radio"/> | <input type="radio"/> | <input type="radio"/> |

In your own words, what's the difference between print and return?

---

What function would you like to create for your own projects?

---

 **Amazing! You can now create your own commands in Python!**